

Relatório - Atividade Prática I

Disciplina: Segurança de sistema de computação

Professor: Ruy de Oliveira

Grupo: Mateus de Souza Arruda e Lorena Strobel Campos

Github: <https://github.com/mateus-s-a/Fundamentos-Hashing-Integridade>

Conceito Teóricos: Fundamentos de Hashing e Integridade de Dados (parte 1)

1.1 Função Hash Criptográfica: O que é uma função hash criptográfica? Cite e detalhe três propriedades essenciais.

Em aplicações de segurança, utilizam-se funções de hash criptográficas. Um algoritmo desse tipo deve garantir que seja computacionalmente inviável encontrar um objeto de dados que seja mapeado para um resultado de hash pré-especificado ou encontrar dois objetos de dados que produzam o mesmo valor de hash. Normalmente, a entrada é preenchida até um múltiplo inteiro de algum tamanho fixo e este inclui o valor do tamanho da mensagem original em bits. Dentre suas propriedades essenciais estão: resistência à pré-imagem, resistência à segunda pré-imagem e resistência à colisão.

A **resistência à pré-imagem**, é a propriedade de mão única, ou seja, é fácil gerar um código a partir da mensagem, porém deve ser praticamente impossível obter uma mensagem a partir de seu código. Se a função hash não possuir essa propriedade de mão única, um invasor pode facilmente descobrir o valor secreto, ou seja, tentar recuperar a mensagem original a partir do seu hash. Sendo assim para qualquer valor de hash h informado, é computacionalmente impossível encontrar y , de modo que $H(y) = h$.

A **resistência à segunda pré-imagem** garante que é impossível encontrar uma mensagem diferente com o mesmo valor de hash. Funciona como prevenção contra a falsificação quando for usado um hash encriptado. Se não for verdadeira, um invasor seria capaz de inicialmente observar ou interceptar uma mensagem com seu código de hash encriptado e posteriormente gerar outra mensagem diferente com o mesmo código de hash encriptado. É essencial para garantir que uma mensagem não seja substituída. Logo para qualquer bloco x informado, é computacionalmente impossível encontrar $y \neq x$ com $H(y) = H(x)$.

A **resistência à colisão forte** caso a função a possua será chamada de função de hash forte, pois protege contra um ataque onde uma terceira parte gera uma mensagem para outra parte assinar, isto é, impossível encontrar duas mensagens diferentes que produzem o mesmo

valor de hash. Em resumo: é computacionalmente impossível encontrar qualquer par (x, y) , de modo que $H(x) = H(y)$.

1.2 Hash vs. Cifragem: Explique a diferença fundamental entre hash e criptografia (cifragem)

Consiste nos esquemas utilizados para a encriptação transformando textos claros (plaintext) em mensagens codificadas os textos cifrados (ciphertext) utilizando uma chave. Para isso, é necessário um algoritmo forte e uma chave com um valor independente do texto claro e do algoritmo. Dessa maneira, a informação que foi cifrada pode posteriormente ser recuperada através do processo de decifração, apanhando o texto cifrado e a chave secreta. Já o hash configura-se como uma mensagem de tamanho variável de produzir um valor de tamanho fixo.

A diferença consiste no objetivo: a criptografia busca a reversibilidade da mensagem, enquanto o hash deve ser computacionalmente inviável obter a mensagem a partir de seu valor de hash, ou seja, deve garantir a confidencialidade da informação, por esse motivo é utilizada para determinar se os dados mudaram ou não.

1.3 Colisões e SHA-1: O que é uma colisão? Por que o algoritmo SHA-1 é considerado obsoleto na segurança moderna?

Uma colisão ocorre se tivermos $x \neq y$ e $H(x) = H(y)$, as entradas são diferentes porém produzem o mesmo valor hash. Como uma das propriedades essenciais é serem resistentes à colisão, espera-se que seja computacionalmente inviável encontrar duas entradas diferentes.

O SHA-1 é um tipo de função hash, sendo SHA a sigla para Secure Hash Algorithm, ele produz um valor de 160 bits que atualmente é considerado obsoleto para aplicações de segurança pois suas resistências a colisões foram quebradas com esforço menor do que o previsto. Desde 2005, o National Institute of Standards and Technology (NIST), agência que desenvolveu o SHA, possuía intenções de retirada da SHA-1, pois, haviam desenvolvido versões com outros tamanho de valor hash. No mesmo ano, pesquisas descreveram um ataque em que duas mensagens separadas poderiam oferecer o mesmo hash SHA-1 usando menos operações do que o esperado. A NIST estabeleceu que até 2031 algoritmos de padrões antigos sejam abandonados.

1.4 Mecanismo de Salt: O que é um "salt" e por que ele é indispensável no armazenamento seguro de senhas?

Consiste em adicionar uma sequência de caracteres única e aleatória a cada senha antes de ela ser criptografada, o “salt” é armazenado juntamente com a senha criptografada. Sua principal vantagem é que cada hash é único mesmo que dois usuários tenham a mesma senha, o que dificulta ataques pré-computados e impede que o mesmo valor de senha gere o mesmo resultado armazenado.

1.5 Tipos de Ataques a Hashes: Diferencie detalhadamente ataque de força bruta, ataque de dicionário e ataque com rainbow tables.

No ataque de força bruta, todas as chaves possíveis são testadas em um trecho do texto cifrado, até obter uma tradução inteligível para o texto claro, em média, metade de todas as possibilidades precisa ser experimentada até encontrar a correta. É preciso haver um certo grau de conhecimento sobre o texto esperado e algum meio de distinguir automaticamente de dados aleatórios.

O ataque dicionário são dicionários de senhas que foram previamente construídas, explorando a criação de senhas por meio de termos do cotidiano e combinações. Além disso, esses também podem ser obtidos a partir de senhas que foram vazadas de diversas organizações ao longo do tempo. Dessa forma, é mais eficiente do que o ataque por força bruta, pois o atacante testa primeiro as possibilidades com maior probabilidade de serem utilizadas.

O ataque de Rainbow tables utiliza tabelas pré-computadas para quebrar os hashes das senhas. Essas tabelas são construídas previamente e armazenam informações que permitem relacionar possíveis textos a seus respectivos hashes. Dessa maneira, o atacante pode comparar um hash obtido de um banco de dados com os valores presentes ou derivados da tabela.

Parte 3: Análise Comparativa

Quantas senhas foram quebradas com sucesso em cada um dos cenários?

Sem salt: Foram fornecidas 17 hashes das quais 15 senhas foram quebradas com sucesso e 2 não foram encontradas por não existirem no dicionário.

Com salt: Das 20 hashes fornecidas, 10 senhas foram quebradas com sucesso e 7 não foram encontradas também por não estarem no dicionário.

Qual o tempo médio de quebra por hash?

A estimativa de tempo foi calculada com os tempos totais de execução capturados pelos scripts utilizando o módulo time do python. Considerando que foram usados 17 hashes:

Sm salt: $0.0261s/17 = 0.0015$ segundos por hash (1,54 milissegundos)

Com salt: $0.0065s/17 = 0.00038$ segundos por hash (0,38 milissegundos)

Explique detalhadamente por que o acréscimo do salt torna o ataque consideravelmente mais lento e menos eficaz:

A adição do salt obriga a combinação da senha com uma sequência de caracteres únicos e aleatórios antes de gerar o hash, apesar de exigirem um tempo maior de processamento para o sistema, promovem uma segurança maior ao tratar cada registro como único e isolar a criptografia. No modelo sem salt, ocorre a comparação somente da senha com o dicionário; logo, senhas iguais podem gerar o mesmo hash, tornando o sistema mais vulnerável.

O que aconteceria se dois usuários tivessem a mesma senha e o mesmo salt? E se tivessem a mesma senha mas salts diferentes?

Dois usuários com a mesma senha e o mesmo salt representam um erro, pois o conceito é serem caracteres únicos ainda que tratem-se da mesma senha, nesse caso os hashes seriam idênticos. Essa situação torna o sistema vulnerável, pois, ao descobrir a senha de um usuário, o atacante descobriria consequentemente a do outro, além da proteção individual ser anulada. A situação onde dois usuários possuem a mesma senha mas salt diferentes e é justamente o propósito do hashes com salt, os hashes finais gerados são completamente diferentes. Dessa maneira, o atacante não consegue estabelecer uma correlação ou matemática entre as contas.

Figura 1: (gerador_dados_teste.py):

```
# hashes COM salt misturando salts reutilizados e salts únicos
salts_reutilizaveis = [os.urandom(16) for _ in range(3)]

# Lista de senhas de teste
senhas_para_salt = [
    "admin", "senha123", "monkey", "football", "123456",
    "root", "master", "shadow", "amor", "matrix", "senha_falsa",
    "usuario1", "usuario2", "usuario3", "usuario4", "usuario5"
]

alvos_com_salt = []
for i, pwd in enumerate(senhas_para_salt):
    # A cada 3 itens, é reutilizado um salt
    if i % 3 == 0:
        salt_escolhido = salts_reutilizaveis[i % len(salts_reutilizaveis)]
    else:
        # Geração de um salt novo
        salt_escolhido = os.urandom(16)

    alvos_com_salt.append((salt_escolhido, pwd))

if i % 4 == 0:
    alvos_com_salt.append((salts_reutilizaveis[0], pwd + "extra"))
```

Figura 2: (quebra_com_salt.py)

```
def quebrar_hashes_com_salt(caminho_hashes, caminho_dicionario):
    registros_alvo = carregar_hashes_com_salt(caminho_hashes)
    senhas_dicionario = carregar_dicionario_senhas(caminho_dicionario)

    # cache de pré-computação por salt: {salt_hex: {hash_gerado: senha}}
    cache_salts = {}
    salts_reutilizados = 0
    quebradas = 0
    nao_encontradas = 0

    for salt_hex, hash_alvo in registros_alvo:
        # Se o salt já foi pré-computado anteriormente no lote, reaproveita o dicionário
        if salt_hex in cache_salts:
            mapa_hash_senha = cache_salts[salt_hex]
            salts_reutilizados += 1
```

Proposta de melhoria: Provolução conceitual para o sistema de cadastro (item 2.3) utilizando um algoritmo de derivação de chave:

Uma evolução para o PBKDF2 (Password-Based Key Derivation Function 2) é o ideal, pois, com os hashes com salt a segurança é grande porém o PBKDF2 transforma chaves maiores e consequentemente geram mais processamento que para o usuário é imperceptível porém é uma diferença muito grande para o invasor. O PBKDF2 aplica a função de hash de forma encadeada milhares de vezes sobre o salt e a senha e para atacante isso multiplica-se exponencialmente no ataque em massa, inviabilizando ataques como o de força bruta.

REFERÊNCIAS:

GOOGLE. **SHattered: The first collision for SHA-1.** Disponível em: <https://shattered.io/pt/colisao-sha1/>. Acesso em: 4 set. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **NIST Retires SHA-1 Cryptographic Algorithm.** 2022. Disponível em: <https://www.nist.gov/news-events/news/2022/12/nist-retires-sha-1-cryptographic-algorithm>. Acesso em: 4 set. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **SP 800-63B: Digital Identity Guidelines.** Disponível em: <https://pages.nist.gov/800-63-4/sp800-63b.html>. Acesso em: 4 set. 2026.

RAMOS, Marlon Brendo. **Redes Neurais Recorrentes para Geração de Senhas em Ataques de Força Bruta baseado em Dicionário.** 2022. Trabalho de Conclusão de Curso (Bacharelado em Sistemas de Informação) — Universidade Federal de Uberlândia, Monte Carmelo, 2022. Disponível em: <https://repositorio.ufu.br/bitstream/123456789/36723/1/RedesNeuraisRecorrentes.pdf>. Acesso em: 4 set. 2026.

TOTVS DEVELOPERS. **Entendendo o PBKDF2: reforçando senhas e chaves com segurança.** Medium, 2020. Disponível em: <https://medium.com/totvsdevelopers/entendendo-o-pbkdf2-refor%C3%A7ando-senhas-e-chaves-com-seguran%C3%A7a-37ab2c8af3c8>. Acesso em: 6 set. 2026.